Universidad Rafael Landívar

Campus San Alberto Hurtado, S.J. de Quetzaltenango

Curso: Programación Web · Docente: Ing. Manuel Rojas

Eventia: de una API insegura a un diseño seguro

Actividad 23 · Entrega completa

Nombre: _____ Carné: _____

Repositorio: https://github.com/carloshugoeg/Eventia

Contenido: auditoría, rediseño, matriz de permisos, OpenAPI, evidencias de controles y preguntas finales.

ENTREGABLE 1

Auditoría de las siete APIs

Caso del enunciado. Riesgos según OWASP API Security Top 10 (2023).

API	Problema identificado	Riesgo OWASP API	Solución propuesta
API 1	GET /api/inscripciones/{id} solo comprueba existencia. No verifica titularidad.	API1 Broken Object Level Authorization (BOLA/IDOR). Secundario: API3.	Autenticación + titularidad o organizador del evento. 403/404 sin filtrar datos ajenos.
API 2	POST /api/usuarios persiste el rol enviado por el cliente (p. ej. administrador).	API3 Broken Object Property Level Authorization (mass assignment). Secundario: API5.	Ignorar rol del cliente. Asignar usuario por defecto. Solo admin puede promover roles.
API 3	POST /api/eventos acepta cupo/precio negativos. Cualquier autenticado puede crear.	API5 Broken Function Level Authorization. Validación de entrada aparte.	Rol organizador. Cupo > 0, precio ≥ 0, título requerido. Rechazar campos extra.
API 4	PUT /api/eventos/{id} exige rol organizador pero no ownership.	API1 Broken Object Level Authorization.	Exigir evento.organizadorId === actor.id. UPDATE condicionado.
API 5	POST inscripciones con cantidad 500 y sin control concurrente de cupos.	API6 Unrestricted Access to Sensitive Business Flows. Secundario: API4.	Tope de cantidad, transacción, UPDATE occupied < capacity.
API 6	GET /api/usuarios/{id} devuelve passwordHash y campos sensibles.	API3 Broken Object Property Level Authorization (exposición excesiva).	DTO mínimo. Nunca devolver hashes. Autorizar vista de perfil.
API 7	Login sin rate limit. Mensajes distintos para correo vs contraseña.	API2 Broken Authentication.	Rate limit, mensaje genérico, comparación timing-safe, hashing lento.

Detalle (problema, consecuencia, OWASP, control)

API 1. Qué está mal: Sin autorización a nivel de objeto. Consecuencia: Fuga de PII y enumeración de asistentes. OWASP: API1. Control: Auth + titularidad.

API 2. Qué está mal: Mass assignment del rol. Consecuencia: Escalada a administrador y bypass de permisos. OWASP: API3. Control: Rol asignado por el servidor.

API 3. Qué está mal: Sin rol ni validación de cupo/precio. Consecuencia: Spam de eventos e integridad rota. OWASP: API5. Control: Organizador + validaciones.

API 4. Qué está mal: Rol sin ownership. Consecuencia: Sabotaje de eventos ajenos. OWASP: API1. Control: Ownership obligatorio.

API 5. Qué está mal: Sin tope ni reserva atómica. Consecuencia: Acaparamiento y sobrecupo. OWASP: API6. Control: Cantidad acotada + SQL condicional.

API 6. Qué está mal: Exposición de passwordHash. Consecuencia: Material para cracking offline. OWASP: API3. Control: Serialización explícita.

API 7. Qué está mal: Fuerza bruta y enumeración. Consecuencia: Compromiso de cuentas y mapa de correos. OWASP: API2. Control: Rate limit + mensaje único.

ENTREGABLE 2

Rediseño de las siete operaciones

Cada operación define método, acceso, datos, validaciones, autorización, respuesta y errores.

1. Consulta de inscripción propia

Endpoint: GET /api/inscripciones/{id} (Eventia: GET /api/registrations)

Función: Listar solo inscripciones del caller.

Quién: Usuario autenticado (titular).

Validaciones: Sesión válida. userId ajeno → 403.

Autorización: Filtrar por actor.id.

Respuestas / errores: 200 { registrations }. 401 / 403.

2. Registro de usuarios

Endpoint: POST /api/usuarios (Eventia: POST /api/auth/register)

Función: Crear cuenta con rol por defecto seguro.

Quién: Visitante.

Validaciones: Nombre, correo, password. Ignorar rol del cliente.

Autorización: Servidor asigna participant/usuario.

Respuestas / errores: 201 perfil público + cookie. 400 / 409 / 429.

3. Creación de eventos

Endpoint: POST /api/eventos (Eventia: POST /api/events)

Función: Publicar evento del organizador.

Quién: Organizador autenticado.

Validaciones: Título, cupo > 0, precio ≥ 0 (si aplica), fecha futura.

Autorización: rol organizador. organizadorId desde sesión.

Respuestas / errores: 201 Created. 400 / 401 / 403.

4. Modificación de eventos

Endpoint: PUT /api/eventos/{id} (Eventia: PATCH /api/events/{id})

Función: Actualizar evento propio.

Quién: Organizador dueño.

Validaciones: Mismas validaciones que creación. Cupo ≥ occupied.

Autorización: rol + organizadorId === actor.id.

Respuestas / errores: 200. 400 / 401 / 403 / 404.

5. Inscripción a un evento

Endpoint: POST /api/eventos/{id}/inscripciones (Eventia: .../registrations)

Función: Reservar plaza del usuario autenticado.

Quién: Usuario registrado.

Validaciones: cantidad ≥ 1 y ≤ máximo (Eventia: solo 1).

Autorización: usuarioId desde sesión. Reserva atómica.

Respuestas / errores: 201. 400 / 401 / 404 / 409.

6. Consulta de perfil

Endpoint: GET /api/usuarios/{id} (Eventia: GET /api/auth/me)

Función: Perfil según nivel de acceso, sin secretos.

Quién: Propietario / admin / vista pública mínima.

Validaciones: id válido.

Autorización: Nunca passwordHash.

Respuestas / errores: 200 campos mínimos. 401 / 403 / 404.

7. Inicio de sesión

Endpoint: POST /api/auth/login

Función: Verificar credenciales e iniciar sesión.

Quién: Visitante.

Validaciones: correo + password. Rate limit.

Autorización: Mensaje genérico de fallo. Cookie HttpOnly.

Respuestas / errores: 200 + cookie. 401 / 429 / 400.

ENTREGABLE 3

Matriz de roles y permisos

P = permitido. D = denegado. Ownership es condición de recurso, no solo de rol.

Operación	Visitante	Usuario registrado	Organizador	Administrador
1. Consultar eventos públicos	P	P	P	P
2. Crear eventos	D	D	P	P (si política)
3. Modificar eventos propios	D	D	P si es dueño	P con auditoría
4. Modificar eventos ajenos	D	D	D	P condicionado
5. Inscribirse en eventos	D	P (sesión+cupo)	P como participante	P si política
6. Consultar inscripciones propias	D	P (solo suyas)	P (solo suyas)	P justificado
7. Inscripciones del evento (organizador)	D	D	P eventos propios	P
8. Administrar usuarios	D	D	D	P

Rol vs recurso. El rol es capacidad global (puede publicar). La autorización de recurso es derecho sobre un objeto concreto (este evento es mío). Fallar solo el rol abre la API 4. Fallar solo el objeto abre la API 1.

En Eventia hoy existen participant y organizer. La columna Administrador es política propuesta.

ENTREGABLE 4

Documentación OpenAPI (tres endpoints)

Spec completo en docs/actividad-23/openapi.yaml. Autenticación: cookie eventia_session. Alineado al rediseño y a la implementación.

PATCH /api/events/{id} — modificar evento propio

Acceso: organizador dueño. Body: title, description, category, location, starts_at, capacity.

200 { event }. 400 (incl. CAPACITY_TOO_LOW). 401. 403 evento ajeno. 404.

POST /api/events/{id}/registrations — inscribirse

Acceso: autenticado. Body JSON obligatorio ({} o { quantity: 1 }). Solo quantity = 1.

201 { registration, notification, event }. 400 INVALID_QUANTITY. 401. 404. 409. 415.

GET /api/registrations — listar propias

Acceso: autenticado. Query userId solo si coincide con la sesión.

200 { registrations } con event anidado. 401. 403 si userId ajeno.

```
openapi: 3.0.3
info:
  title: Eventia API (endpoints corregidos)
  version: 1.0.0
paths:
  /api/events/{id}:
    patch:
      security: [{ sessionCookie: [] }]
      summary: Modificar un evento propio
  /api/events/{id}/registrations:
    post:
      security: [{ sessionCookie: [] }]
      summary: Inscribirse en un evento
  /api/registrations:
    get:
      security: [{ sessionCookie: [] }]
      summary: Listar inscripciones propias
components:
  securitySchemes:
    sessionCookie:
      type: apiKey
      in: cookie
      name: eventia_session
```

ENTREGABLE 5

Evidencias de dos controles implementados

Control 1. Ownership al modificar eventos (API 4)

1. Comportamiento anterior. Un PATCH/PUT que solo exigía rol organizador permitía alterar eventos ajenos.

2. Problema. OWASP API1 Broken Object Level Authorization.

3. Modificación. PATCH /api/events/:id. events.update exige organizer_id === user.id. UPDATE con WHERE id AND organizer_id. UI solo al dueño.

4. Prueba válida. Organizador A hace PATCH de su evento → 200, título actualizado.

5. Prueba no autorizada. Organizador B hace PATCH al mismo id → 403 FORBIDDEN.

6. Resultado. Solo el dueño modifica. Cubierto por npm test (fase 3 ownership / HTTP PATCH ajeno).

Control 2. Cantidad de inscripción y cupos (API 5)

1. Comportamiento anterior. cantidad: 500 sin tope y sin reserva concurrente segura.

2. Problema. OWASP API6 (principal); API4 secundario.

3. Modificación. quantity entero ≥ 1 y solo 1. reservePlace si available ≥ 1. SQL occupied < capacity en transacción BEGIN IMMEDIATE. Concurrencia 8 vs 3 plazas.

4. Prueba válida. POST .../registrations con {} o { quantity: 1 } → 201.

5. Prueba inválida. quantity 0/-1/2 → 400 INVALID_QUANTITY. Tercera plaza con cupo 2 → 409 EVENT_FULL.

6. Resultado. Backend rechaza abuso y mantiene cupo. Cubierto por npm test.

Reproducción: npm test · Casos fase 3 ownership, quantity y HTTP rechazo.

ENTREGABLE 6

Enlace al repositorio

https://github.com/carloshugoeg/Eventia

Paquete de la Actividad 23 en docs/actividad-23/. Controles en código y pruebas en tests/eventia.test.js.

Verificación: node scripts/check-actividad-23.js && npm test

ENTREGABLE 7

Respuestas a las preguntas finales

1. ¿Cuál de las siete APIs representa el mayor riesgo?

La API 2 (registro con rol elegido por el cliente). Si cualquiera se autoasigna administrador, el resto de controles de rol dejan de importar. API 1 y API 6 son graves en confidencialidad; API 5 afecta integridad del negocio. Ninguna iguala el alcance de esa elevación de privilegios.

2. ¿Por qué JWT no garantiza que una API esté protegida?

JWT transporta identidad (y claims). No sustituye autorización por recurso, validación, rate limiting, minimización de datos ni reglas de negocio. Un JWT con rol organizador sigue pudiendo editar un evento ajeno si el backend no comprueba organizadorId. El token autentica; la política se implementa en el servidor.

3. ¿Diferencia entre validar datos y autorizar una operación?

Validar: ¿el cuerpo es sintáctica y semánticamente correcto? (cupo entero positivo, quantity ≥ 1). Autorizar: ¿este actor puede hacer esta acción sobre este recurso? Un request puede ser válido y estar prohibido (403), o autorizado y fallar validación (400). Ambos son necesarios.

4. ¿Por qué documentar autenticación en OpenAPI no implementa seguridad?

OpenAPI describe el contrato. No ejecuta comprobaciones. Un spec puede marcar security mientras el handler omite requireUser u ownership. La seguridad existe cuando el código y las pruebas rechazan los casos inválidos.

5. ¿Qué controles mantener con servicios independientes?

- Autenticación centralizada en un borde confiable.
- Autorización en cada servicio que posea el recurso.
- Validación en el borde de cada servicio.
- Identidad del caller propagada de forma confiable (claims firmados / mTLS).
- Límites de tasa en login e inscripción.
- Transacciones o sagas para cupos compartidos.
- Minimización de datos en respuestas y logs.
- Auditoría de acciones privilegiadas.
- Secretos fuera del código y TLS entre clientes y servicios.